

3

Interim Report

3.1 Progress

Currently, we have finished the functional minimum and the low target of our proposed game. The player can now use both the melee and the ranged weapon to engage in combats. The pictures below show the combat experience. In particular, the player launches a melee attack with Left Mouse Button or RB on the Xbox controller; fires the pistol with Right Mouse Button or LB on the Xbox controller. Both of these attacks are aimed either with the mouse cursor, or with the Right Thumb Stick on controller.



(a) *Melee Attack*



(b) *Ranged Attack*

Figure 3.1: Available Player Attacks



(a) Slime



(b) Goblin

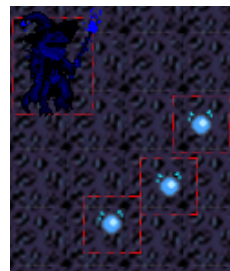


(c) Rat Swordsman

Figure 3.2: Examples of melee enemies



(a) Lizard Mage



(b) Lizard Archmage

Figure 3.3: Examples of ranged enemies

We have also implemented various enemies with differentiated AI behaviors. Specifically, we have three melee enemies and two ranged enemies. Melee enemies attack the player by getting close. The slime deals damage by bumping into the player, the goblin uses a dagger to slash the player, and the rat swordsman wields a sword like the player does. The AI behaviors behind them specialize to fit the enemies' attack patterns. In addition, the ranged enemies shoot projectiles toward the player and runs away if the play gets too close. There are currently two types of lizard mage as ranged enemies in our game. One of them fires a single shot at a time, and the other one shoots three projectile to different directions like a shotgun.

We also made progress in random map generation. Given the lack of support for dynamic map generation during runtime from the MonoGame.Extended.Tiled library, we wrote our own tool that interprets and renders a .tmx file generated from Tiled. Using the said tool, we currently have several levels with randomized enemies spawning at set positions. The player can progress through these levels without the option to backtrack.

For art coherence, we created animations for the player, lizard mage, and the rat swordsman. Although the art assets of the slime and the goblin are from the Internet currently, we have plans to replace them with assets created by us in the future. We also added sound effects to the game such as the sound of sword swinging, shots firing, and getting hurt.

We are currently beginning to work on our desired targets. Some basic power-ups have already been implemented, which act as simple stat improvements for the player.

3.2 Challenges

One of the biggest challenges we faced was in trying to use the MonoGame.Extended.Tiled library for random map generation. The way this library was implemented made any edits to rendered maps during runtime impossible, so we either had to pre-generate a large number of maps, or write our own map renderer. We chose the latter option, and learnt that we actually have a lot more freedom in map generation than we initially thought, as we are now free to implement any map-related functionalities as we see fit.

Another unexpected challenge was in asset generation. While we thought creating enemy sprites for 6 to 8 directions of orientation would be relatively easy, this turned out to be significantly harder and time-consuming than what we initially expected. After some testing, we collectively decided that only supporting left and right enemy orientations looked acceptable. We may revisit this later in development and add additional sprite sets if we have enough time. For now, we decided to avoid designing enemies that need more than two orientations to not look awkward, like a sniper that visibly aims towards the player.

3.3 Future Work

We plan to finish off our desired targets as a priority. We will implement and polish a parry mechanic and add a wider array of power-ups to the game. We are currently revising whether to implement the ability to swap between one-handing and two-handing the player's primary weapon, as we have become unsure if this functionality offers meaningful improvement to the combat feel.

As we work on ticking off our desired targets, we will also make small adjustments and additions to aspects we already implemented for the interim demo, like adding more enemy types or polishing the combat system to feel more fluid, especially with the addition of the parry mechanic.

3.4 Contributions

Alkaya Ömer	Implemented the overall map generation logic and designed levels.
Willy Shih	Implemented ranged projectiles and powerups.
Yingzhe Liu	Made assets for player and enemy, added sound effects. Contributed to the report.
Minsung Kang	Implemented ranged enemies and basic entity spawn logic. Contributed to the report.
Shenghao Zhang	Implemented slime-type enemy, camera, level progression and UI.